



Fakultet informatike u Puli

student
Mario Bariša

Analiza popularnosti Spotify pjesama

Izvještaj o projektnom zadatku

SEMINARSKI RAD

U Puli, 4. lipnja 2026.godine
za kolegij "Skladišta i rudarenje podataka"
Profesor: izvanredni profesor Goran Oreški

IZJAVA O AKADEMSKOJ ČESTITOSTI

Ja, dolje potpisan Bariša Mario, ovime izjavljujem da je ovaj seminarski rad rezultat isključivo mogega vlastitog rada, da se temelji na mojim istraživanjima te da se oslanja na objavljenu literaturu kao što to pokazuju korištene bilješke i bibliografija. Izjavljujem da niti jedan dio seminarskog rada nije napisan na nedozvoljen način, odnosno da je prepisan iz kojega necitiranog rada, te da ikoji dio rada krši bilo čija autorska prava. Izjavljujem, također, da nijedan dio rada nije iskorišten za koji drugi rad pri bilo kojoj drugoj visokoškolskoj, znanstvenoj ili radnoj ustanovi.

A handwritten signature in black ink, appearing to read 'Mario Bariša', written in a cursive style.

SADRŽAJ

1. Uvod u tematiku i problematiku, cilj analize dataseta-a	1
2. Izrada relacijskog modela	3
3. Izrada dimenzijskog modela	5
4. ETL Proces - Opis procedure, od početnog učitavanja do inserta	7
5. Analiza podataka	12
6. Zaključak	21
7. Literatura	22

1. Uvod u tematiku i problematiku, cilj analize dataseta-a

Poslovni svijet kakvog znamo u današnjem svijetu izuzetno je konkurentan i kompetitivan, a svaki detalj, ma koliko malen, bitan je kako bi neki pojedinac i/ili poslovni subjekt bio u boljoj poziciji naspram drugog. Od umjetne inteligencije do tržišne analize, tvrtke gledaju sve moguće kvalitetne izvore informiranja kako bi mogle provoditi dobre i vremenski relevantne poslovne odluke u svrhu maksimiziranja profita i zadovoljstva klijenata.

Prvi korak u projektnom ciklusu bio je pronaći neki dobar dataset nad kojim ću moći raditi željenu vrstu analize. Nakon duge potrage pronašao sam “spotify-tracks-dataset” (link na navedeni nalazi se u README.md). Uz vrlo detaljno razrađene detalje o pjesmama – bili oni tehničke ili neke druge prirode – navedeni mi se svidio jer je fokus dataseta bio oko indeksa popularnosti te sam odmah uočio da ovdje mogu napraviti zanimljivu i korisnu analizu. Povezivanjem tehničkih detalja i onih manje tehničkih mogu na kraju analize povući crtu te reći što generalno prolazi najbolje kod opće publike.

GENERALNI UVJETI ZA DATASET:

1. Skup podataka mora imati 20 ili više stupaca
2. Skup podataka mora imati više od 15 000 redaka, a idealna količina
3. Redaka je između 50 000 i 150 000
4. Skup podataka treba imati 5 dimenzija, jedna od kojih mora (poželjno je) biti vremenska dimenzija
5. Skup bi trebao imati kvalitativne i kvantitativne podatke
6. Skup bi trebao imati što manje nepostojećih vrijednosti

Generalni pregled dataset-a

BROJ STUPACA	BROJ REDOVA	REDOVI NAKON ČIŠĆENJA	80 %	20 %	Vremenska dimenzija?
21	114 000	113 549	90839	22710	NE -> dodano kasnije radi primjera

Zbog svega navedenog, ja sam u sklopu projektnog zadatka odlučio gledati analizu najpopularnijih Spotify pjesama (snapshot 2023. godine) kako bih mogao fiktivnom glazbeno-producentnom studiju "Studio d.o.o." dati informacije i detalje: koje su pjesme najpopularnije i zašto, te možemo li nekako presjeći paralelu i generalizirati omjer popularnosti s različitim faktorima pjesama – od onih tehničkih do psiholoških. Cilj je da studio može donekle garantirati da će pjesme koje se kod njih budu izrađivale biti u skladu s provedenom analizom te na taj način garantirati (u određenoj mjeri) popularnost istih.

Alati i tehnologije koje sam koristio tijekom izrade bili su VS Code (Jupyter notebook), DataGrip (MySQL IDE), MySQL Server 8, MySQL Workbench, MermaidChart.js za izradu dijagrama, Apache Spark te Docker + MetaBase za izradu dashboarda.

2. Izrada relacijskog modela

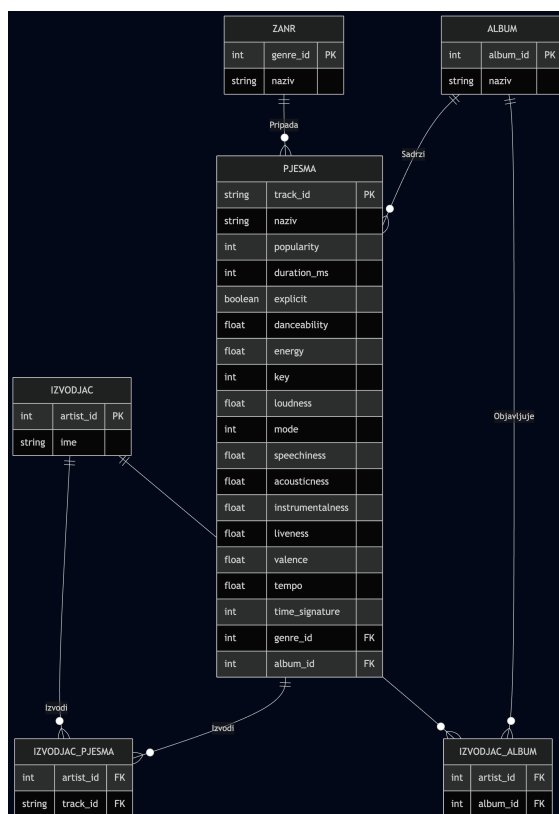
Kako je sam cilj projekta izraditi dashboard putem ETL Spark frameworka, prvi korak nakon pronalaska dataseta je bilo navedeni pretočiti u relacijski model. Za potrebe izrade dijela transformacijskog procesa korištene su polazne skripte dostupne na javnom repozitoriju, koje su potom prilagođene i nadopunjene prema specifičnostima odabranog dataseta i zahtjevima projektnog zadatka. Navedene skripte služile su kao osnova, dok su ključne izmjene, logika obrade i integracija podataka provedene samostalno. Iz dataseta su se prvo izbacili svi NULL podaci, a zatim se podijelio 80/20 kako bi se kasnije ostatak mogao učitati u Spark Framework.

Tablice u modelu su:

1. Pjesma (najbitnija; sadrži sve informacije o njezinim karakteristikama)
2. Album (pjesma mora pripadati albumu; jedan album ima više pjesama, ali svaka pjesma je u samo jednom albumu)
3. Žanr (vrlo bitna tablica jer govori što je pjesma sama po sebi, te je bitan podatak za kasniju kategorizaciju)
4. Izvođač (tko pjeva?)

Pomoćne tablice u modelu su:

1. Izvođač_pjesma (jer jedan izvođač gotovo uvijek ima više pjesama)
2. Izvođač_album (izvođač gotovo uvijek ima više od jednog albuma)



Tablica "Pjesma" je najbitnija jer se u njoj nalaze sve najbitnije informacije, dok su ostale tablice manje bitne te je podjela napravljena na ovaj način kako bi se napravio jasan presjek između akustičnih informacija pjesme od onih koje to nisu.

Navedeni model je tada pretočen u klasične SQL insert naredbe te je još dodatno napravljena SQL SELECT provjera da se vidi poklapa li se broj redova, te da nije došlo do gubitka podataka tijekom transformacije.

3. Izrada dimenzijskog modela

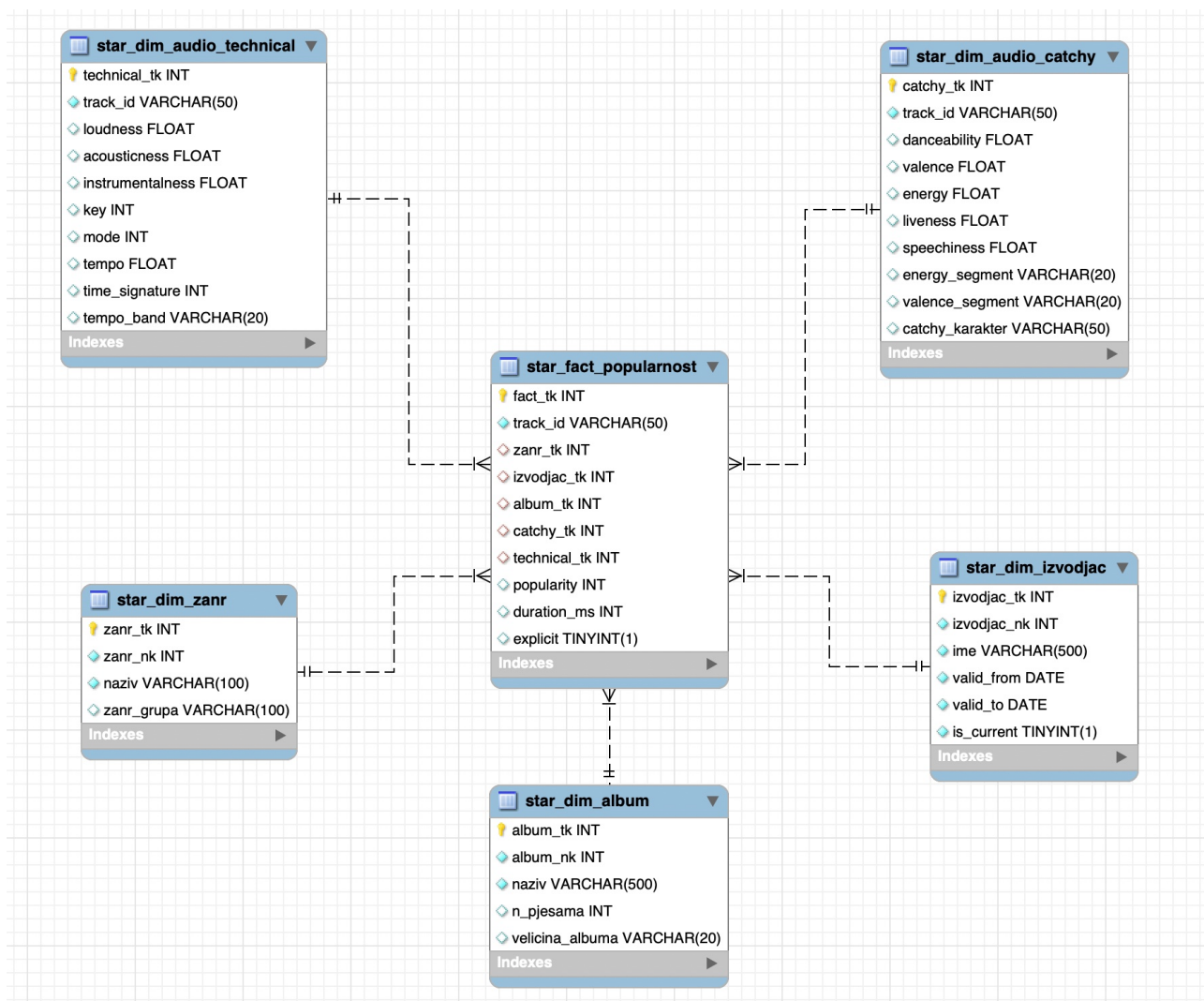
Tijekom prethodnih akademskih godina detaljno smo učili kako se gradi relacijski model te koja mu je svrha i uloga u praksi. Međutim, za cilj projektnog zadatka potrebno je izraditi dimenzijski model (STAR). Prvi i ključni korak pri izgradnji dimenzijskog modela jest prepoznavanje tablice činjenica. U ovom slučaju, tablica činjenica odnosi se na popularnost pjesama, jer je upravo to glavni cilj analize. Stoga sam odlučio da će upravo taj podatak biti najvažniji u cijelom modelu.

OPIS DIMENZIJA

1. `star_fact_popularnost` - Ujedno i tablica činjenica. Ova tablica je izuzetno bitna jer sadrži glavni faktor kasnije analize, a to je indeks popularnosti pjesme. Kao takva, ova tablica je najbitnija jer će se sve analize protezati kroz nju.
2. `star_dim_catchy` - Ova dimenzijska tablica predstavlja sve netehničke detalje pjesme. Iako nisu tehnički detalji pjesme, i dalje su izuzetno korisni podaci jer govore i opisuju na koji način pjesma utječe na publiku. Npr. "valence_segment" opisuje (od 0 do 1) koliko je pjesma "pozitivna" te kako se taj faktor prelijeva u ostale faktore i u samom koncu na popularnost.
3. `star_dim_audio_tehcnical` - Dimenzijska tablica opisuje čiste tehničke karakteristike pjesme koje mogu dobro poslužiti producentima u stvaranju djela. Na primjer, faktor "Tempo" govori o tome kakav je BPM, točnije kakav će ritam pjesma imati – jedna od najbitnijih karakteristika pjesme.
4. `star_dim_zanr` - Tablica opisuje kakvog je žanra pjesma (od onih 110+) te kojoj žanr grupi pripada (za lakše generaliziranje).
5. `star_dim_album` - Tablica sadrži naziv albuma, broj pjesama u albumu i veličinu navedenog.
6. `star_dim_izvodjac` - Tablica s popisom svih izvođača prikupljenih pjesama. Dodana je vremenska dimenzija u sklopu aktivnosti umjetnika; svi umjetnici su stavljeni da su

aktivni jer je dataset snapshot iz 2023. kada su svi navedeni bili aktivni. Vremenska dimenzija se nije kasnije koristila za analizu te ista nije potrebna. Navedena je ovdje samo zbog ispunjenja zahtjeva programskog zadatka.

Vizualni prikaz dimenzijskog modela



4. ETL Proces - Opis procedure, od početnog učitavanja do inserta

ETL proces predstavlja i najvažniji i najzahtjevniji dio cijelog projekta. Iako se sastoji od mnogo pojedinačnih elemenata koji moraju funkcionirati nezavisno, na kraju ih je potrebno uskladiti u jedinstvenu cjelinu kako bi čitav proces završio uspješno. U ovom projektu izdvajanje (ekstrakcija) podataka odvija se iz dva različita izvora: iz CSV datoteke i iz MySQL baze podataka, i to u omjeru 80 % prema 20 %. Tijekom faze izdvajanja posebnu pozornost treba obratiti na očuvanje oblika koju smo definirane prilikom dizajniranja dimenzijskog modela – taj oblik mora ostati nepromijenjen.

Nakon što su podaci izvučeni, slijedi faza transformacije koja predstavlja najzahtjevniji dio cijelog ETL procesa. Glavni izazov leži u činjenici da izvorni podaci dolaze u različitim formatima – MySQL relacijske tablice i CSV datoteka – a svaki od njih ima vlastitu strukturu, nazive atributa i tipove podataka. Kako bi se podaci mogli spojiti u jedinstveni dimenzijski model, bilo je potrebno izvršiti normiranje naziva stupaca, usklađivanje tipova podataka te mapiranje vrijednosti.

U ovom projektu svaka dimenzijska tablica transformirana je zasebno, mislim da je ovakav pristup nešto kompleksniji od all in one ali ga je kasnije lakše shvatiti kako i na koji način je implementiran. Posebna pažnja posvećena je spajanju podataka iz MySQL-a i CSV-a – za svaku dimenziju najprije su obrađeni podaci iz baze, zatim su dodani oni iz CSV datoteke koji nisu bili prisutni u bazi, te su na kraju svi zapisi objedinjeni u jedinstvenu dimenzijsku tablicu. Ovaj proces dodatno je proširen implementacijom mehanizma za upravljanje sporo mijenjajućim dimenzijama (takozvani SCD tip 2) za tablicu izvođača, što omogućuje praćenje povijesnih promjena naziva izvođača, iako kao što sam prije naveo ovo nije bilo implementirano u analizi kasnije jer problematika koju istražujem nije povezana sa tim činjenicama. Tu je samo kao dokaz mog razumijevanja navedene implementacije vremenske dimeznije.

Za tablicu činjenica proces je nešto složeniji jer se uz usklađivanje naziva i tipova podataka moraju precizno povezati i strani ključevi prema svim dimenzijskim tablicama. Svaka veza između činjenice i dimenzije detaljno je definirana i implementirana kako bi se osigurala referencijalna cjelovitost podataka. U nastavku je prikazan Python kod korišten za transformaciju, a slike 14–18 ilustriraju ključne korake ovog postupka.

transform_zanr_dim – grupiranje žanrova u kategorije i stvaranje dimenzije žanra

```
def transform_zanr_dim(zanr_df, csv_df=None):
    max_zanr_nk = (
        zanr_df
        .select(col("id").cast("int").alias("id"))
        .groupBy()
        .max("id")
        .collect()[0]["max(id)"]
    ) or 0

    df = (
        zanr_df
        .select(col("id").alias("zanr_nk"), trim(col("naziv")).alias("naziv"))
        .dropDuplicates(["naziv"])
        .fillna({"naziv": "Unknown"})
    )

    if csv_df is not None:
        # extract zanrova iz CSV koji mozda nisu u trenutnom DIM
        csv_zanr = (
            csv_df
            .select(trim(col("track_genre")).alias("naziv"))
            .distinct()
            .withColumn("zanr_nk", lit(None).cast("int"))
        )
        df = df.unionByName(csv_zanr).dropDuplicates(["naziv"])

    nk_window = Window.orderBy("naziv")
    df = (
        df
        .withColumn("_rn_nk", row_number().over(nk_window))
        .withColumn(
            "zanr_nk",
            when(col("zanr_nk").isNull(), lit(max_zanr_nk) + col("_rn_nk"))
            .otherwise(col("zanr_nk").cast("int"))
        )
        .drop("_rn_nk")
    )

    df = df.withColumn("zanr_grupa", zanr_grupa_udf(col("naziv")))
    window = Window.orderBy("naziv")
    df = df.withColumn("zanr_tk", row_number().over(window))
    return df.select("zanr_tk", "zanr_nk", "naziv", "zanr_grupa")
```

Kako je bilo 110+ originalnih žanrova koji realno gledajući nisu potrebni za donošenje krajnje odluke / presude moje analize, radi lakšeg praćenja odlučio sam implementirati znar_ grupe. Ovdje umjesto analize po 110 različitih žanrova oni se generaliziraju u

nekoliko sličnih krupa kako bi se mogala dobiti relevantna slika kako koja žarna grupa prolazi kod publike. Ovo zadovoljava moji krajnji cilj.

Isječak iz koda gdje se definiraju žanr grupe prema generalno “najbitnijim” žanrovima:

```
def _zanr_grupa(naziv):
    if naziv is None:
        return "Other"
    n = naziv.lower()
    if any(x in n for x in ["acoustic", "folk", "singer", "indie", "alt-country", "country"]):
        return "Chill/Acoustic"
    if any(x in n for x in ["pop", "power-pop", "synth-pop", "cantopop", "mandopop", "k-pop", "j-pop"]):
        return "Pop"
    if any(x in n for x in ["hip-hop", "rap", "trap", "r-n-b", "soul", "funk"]):
        return "Urban/R&B"
    if any(x in n for x in ["rock", "metal", "punk", "grunge", "garage", "hard-rock", "psych"]):
        return "Rock/Metal"
    if any(x in n for x in ["dance", "edm", "electro", "house", "techno", "trance", "dubstep", "drum"]):
        return "Electronic/Dance"
    if any(x in n for x in ["jazz", "blues", "classical", "piano", "opera", "show-tunes"]):
        return "Classic/Jazz"
    if any(x in n for x in ["latin", "salsa", "samba", "mpb", "pagode", "sertanejo", "axe", "forro"]):
        return "Latino/World"
    return "Other"
```

transform_izvodjac_dim – SCD tip 2 obrada podataka o izvođačima

```
def transform_izvodjac_dim(izvodjac_df, csv_df=None):
    max_izvodjac_nk = (
        izvodjac_df
        .select(col("id").cast("int").alias("id"))
        .groupBy()
        .max("id")
        .collect()[0]["max(id)"]
    ) or 0

    df = (
        izvodjac_df
        .select(col("id").alias("izvodjac_nk"), trim(col("ime")).alias("ime"))
        .dropDuplicates(["ime"])
        .fillna({"ime": "Unknown Artist"})
    )

    if csv_df is not None:
        csv_izv = (
            csv_df
            .select(explode(split(col("artists"), ";")).alias("ime"))
            .select(trim(col("ime")).alias("ime"))
            .distinct()
            .withColumn("izvodjac_nk", lit(None).cast("int"))
        )
        df = df.unionByName(csv_izv).dropDuplicates(["ime"])

    nk_window = Window.orderBy("ime")
    df = (
        df
        .withColumn("_rn_nk", row_number().over(nk_window))
        .withColumn(
            "izvodjac_nk",
            when(col("izvodjac_nk").isNull(), lit(max_izvodjac_nk) + col("_rn_nk"))
            .otherwise(col("izvodjac_nk").cast("int"))
        )
        .drop("_rn_nk")
    )

    df = (
        df
        .withColumn("valid_from", current_date())
        .withColumn("valid_to", lit("9999-12-31").cast("date"))
        .withColumn("is_current", lit(True).cast(BooleanType()))
    )

    window = Window.orderBy("ime")
    df = df.withColumn("izvodjac_tk", row_number().over(window))
    return df.select("izvodjac_tk", "izvodjac_nk", "ime", "valid_from", "valid_to", "is_current")
```

transform_audio_catchy_dim – obrada catchy audio značajki

```
def transform_audio_catchy_dim(pjesma_df, csv_df=None):
    def _select_catchy(df):
        return df.select(
            col("track_id"),
            col("danceability").cast("double"),
            col("valence").cast("double"),
            col("energy").cast("double"),
            col("liveness").cast("double"),
            col("speechiness").cast("double")
        )

    df = _select_catchy(pjesma_df)

    if csv_df is not None:
        df = df.unionByName(_select_catchy(csv_df))

    df = (
        df
        .dropDuplicates(["track_id"])
        .dropna(subset=["track_id"])
        .withColumn("energy_segment", energy_udf(col("energy")))
        .withColumn("valence_segment", valence_udf(col("valence")))
        .withColumn("catchy_karakter", catchy_udf(col("energy_segment"), col("valence_segment")))
    )

    window = Window.orderBy("track_id")
    df = df.withColumn("catchy_tk", row_number().over(window))

    return df.select(
        "catchy_tk", "track_id", "danceability", "valence",
        "energy", "liveness", "speechiness",
        "energy_segment", "valence_segment", "catchy_karakter"
    )
```

transform_audio_technical_dim – obrada tehničkih audio značajki

```
def transform_audio_technical_dim(pjesma_df, csv_df=None):
    def _select_technical(df, id_col):
        return df.select(
            col(id_col).alias("track_id"),
            col("loudness").cast("double"),
            col("acousticness").cast("double"),
            col("instrumentalness").cast("double"),
            col("key").cast("int"),
            col("mode").cast("int"),
            col("tempo").cast("double"),
            col("time_signature").cast("int")
        )

    df = _select_technical(pjesma_df, "track_id")

    if csv_df is not None:
        csv_tech = _select_technical(csv_df, "track_id")
        df = df.unionByName(csv_tech)

    df = (
        df
        .dropDuplicates(["track_id"])
        .dropna(subset=["track_id"])
        .withColumn("tempo_band", tempo_udf(col("tempo")))
    )

    window = Window.orderBy("track_id")
    df = df.withColumn("technical_tk", row_number().over(window))
    return df.select(
        "technical_tk", "track_id", "loudness", "acousticness",
        "instrumentalness", "key", "mode", "tempo", "time_signature", "tempo_band"
    )
```

Kada govorimo o transformaciji specifičnih informacija o pjesmi, oko tehničkih detalja nije bilo puno posla – ti su podaci već dobro poznati svim producentima i glazbenim stručnjacima. No, s takozvanim "catchy" podacima stvar stoji malo drugačije. Budući da su ti podaci u datasetu bili definirani na način prilagođen isključivo tom skupu podataka, trebao sam ih pretvoriti u oblik koji će svatko moći razumjeti, umjesto da ostanu kao obične *"plain"* brojčane vrijednosti. Zbog toga sam implementirao dvije pomoćne funkcije kako bih to i postigao.

Helper funkcije za catchy tablicu

```
def _energy_segment(e):  
    if e is None: return "unknown"  
    if e < 0.33: return "low"  
    if e < 0.66: return "mid"  
    return "high"  
  
def _valence_segment(v):  
    if v is None: return "unknown"  
    if v < 0.33: return "negative"  
    if v < 0.66: return "neutral"  
    return "positive"
```

Posljednji korak ETL procesa jest učitavanje transformiranih podataka u odredišnu bazu podataka. U ovoj fazi podaci se preuzimaju iz privremenog skladišnog područja (rezultat transformacije) i upisuju u novu MySQL bazu podataka strukturiranu prema star shemi. Kod za umetanje podataka napisan je u Pythonu, a njegova je logika relativno jednostavna. Prvo se uspostavlja veza s odredišnom bazom, zatim se brišu postojeći podaci iz svih tablica (TRUNCATE) kako bi se osiguralo čisto okruženje za upis, a potom se svaka transformirana tablica upisuje u odgovarajuću tablicu. Prije samog upisa onemogućuje se provjera stranih ključeva kako bi se izbjegli sukobi prilikom redoslijeda upisa, a nakon završetka upisa provjera se ponovno uključuje. Nakon završetka umetanja izvršio sam provjeru redova putem običnog SELECT i provjerio poklapa li se sa početnim brojev redova nakon čišćenja.

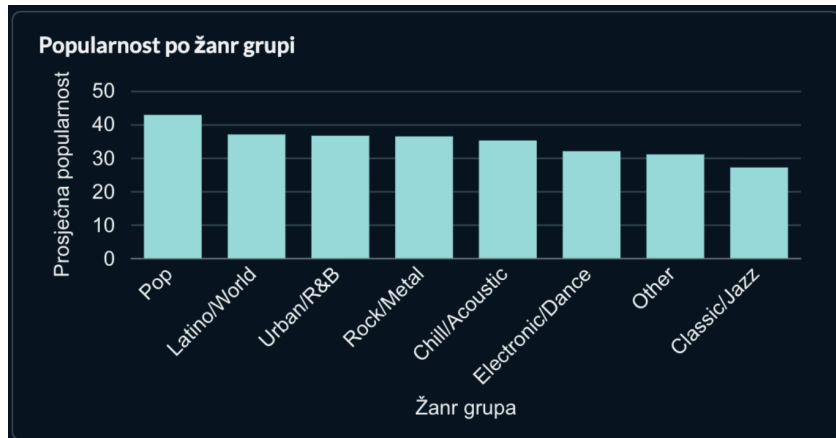
5. Analiza podataka

Nakon što su podaci učitani u MySQL bazu, potrebno ih je vizualizirati. Postoji mnogo vizualizacijskih alata, primjerice Tableau. Za ovaj projekt korišten je Metabase – FOSS (free and open-source) alat.

Metabase je pokrenut putem Dockera kako bi se jednostavno instalirao i pokrenuo na macOS-u. Prvo se unutar aplikacije treba spojiti na MySQL bazu podataka u kojoj se nalaze dimenzijske tablice i tablica činjenica. Svaka se tablica u Metabaseu vodi kao zaseban skup podataka „*database*“.

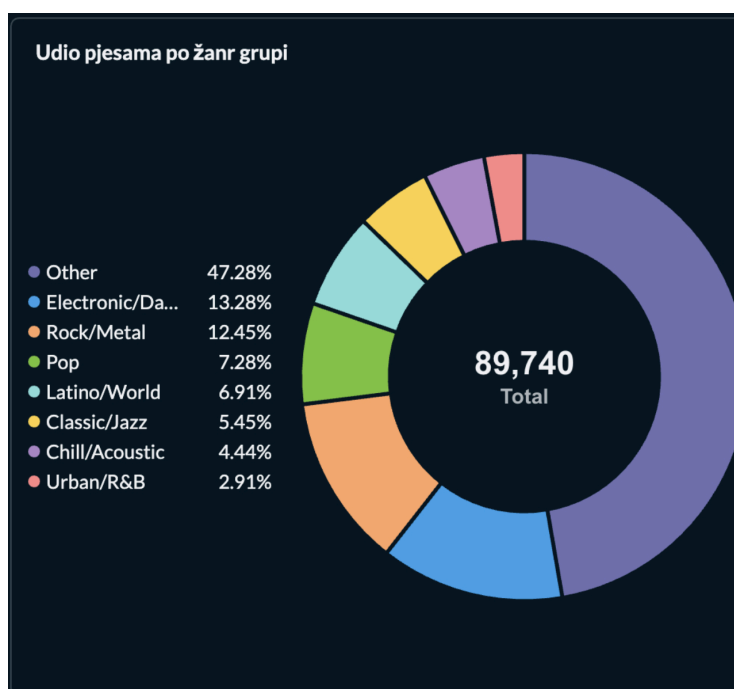
Metabase grafički prikazuje podatke izravno nad bazom, ali također omogućuje pisanje vlastitih SQL upita koji koriste podatke iz više tablica odjednom. Rezultat upita Metabase zatim tretira kao novi privremeni skup podataka na temelju kojeg se crtaju grafikoni. Na taj su način izrađeni svi potrebni dashboardovi za ovaj projekt. Ja sam svoje grafove izradio pisanjem SQL upita jer mi je isto bilo najlakše.

Popularnost po žanr grupama



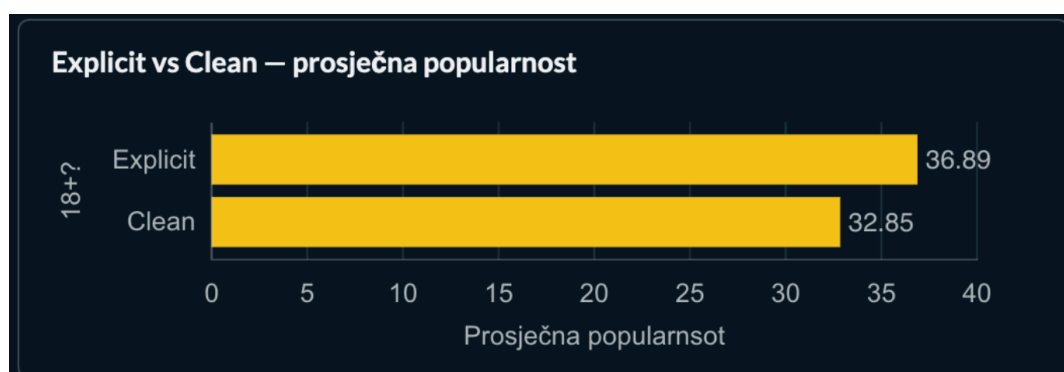
Korišten je horizontalni bar chart koji prikazuje popularnost za svaku žanr grupu. Kao što sam prije naveo žanr grupe sam napravio s ciljem lakšeg i bržeg razumijevanja grafova te iz razloga što sve svi žanrovi u određenog žanr grupi dosta miješaju. Može se vidjeti da je žanr grupa Pop najpopularnija, dok je Classic/Jazz najmanje popularna. Ovo je očito jer mlađe generacije koje pretežito koriste platforme poput Spotify-a ne slušaju “dosadne” pjesme.

Udio pjesama po žanr grupama



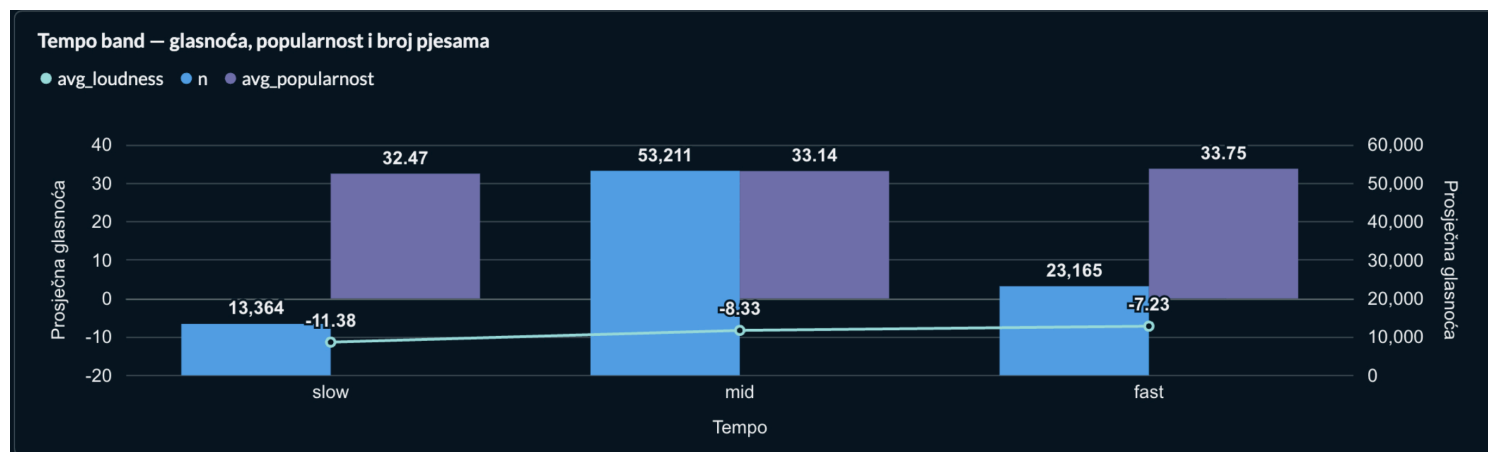
Korišten je pie chart koji prikazuje postotni udio svake žanr grupe u ukupnom broju pjesama. Može se vidjeti da grupa *Other* dominira s 47,28%, dok je Electronic/Dance najzastupljenija s tek 13.3%. Ovo daje zanimljivu perspektivu na cijelu priču jer iako nam prvi graf govori da je Pop najpopularniji sam broj pjesama iz tog žanra nije toliko veliki u usporedbi sa drugima, točnije iako Pop ima manje pjesama veliki broj tih pjesama bude izrazito popularan. Veliko odudaranje od ostalih žanr grupa. Grupa “*Other*” je zapravo skup svih žanrova koji u trenutku *snapshota* dataseta bili popularni ali nisu dovoljno bitni za ulazak u samu analizu iako ih ima puno njihova popularnost ne garantira popularnos sutra.

Prosječna popularnost prema vrsti sadržaja (NSFW vs SFW)



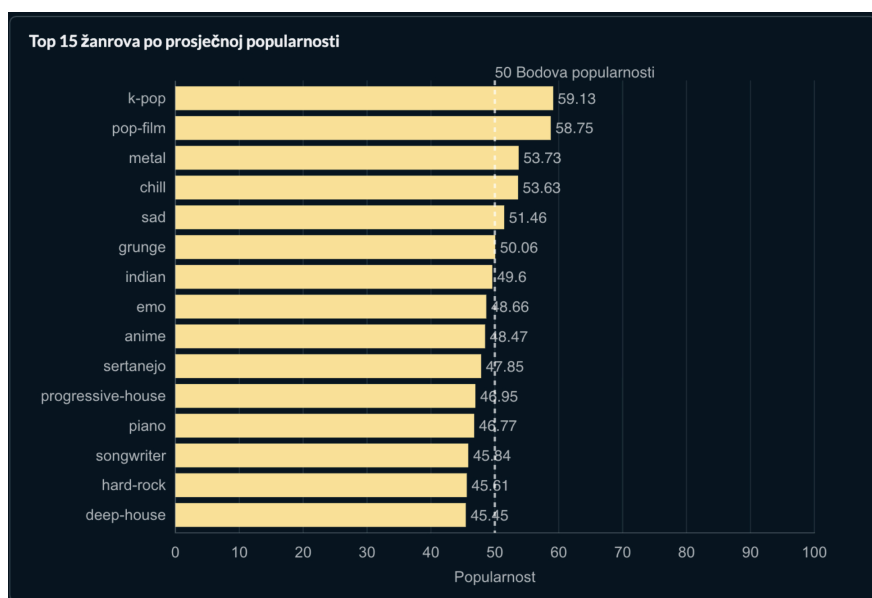
Korišten je horizontalni bar chart koji uspoređuje prosječnu popularnost pjesama s eksplicitnim sadržajem i onih bez njega. Može se vidjeti da pjesme označene kao Explicit imaju nešto veću prosječnu popularnost (36,89) u odnosu na Clean (32,85). Iako je razlika uočljiva ista ne garantira uspješnost pjesme te se ne preporučuje da pjesma bude prosta bez potrebe ukoliko to nije eksplicitni uvjet umjetnika.

Odnos tempa, glasnoće, broja pjesama i popularnosti



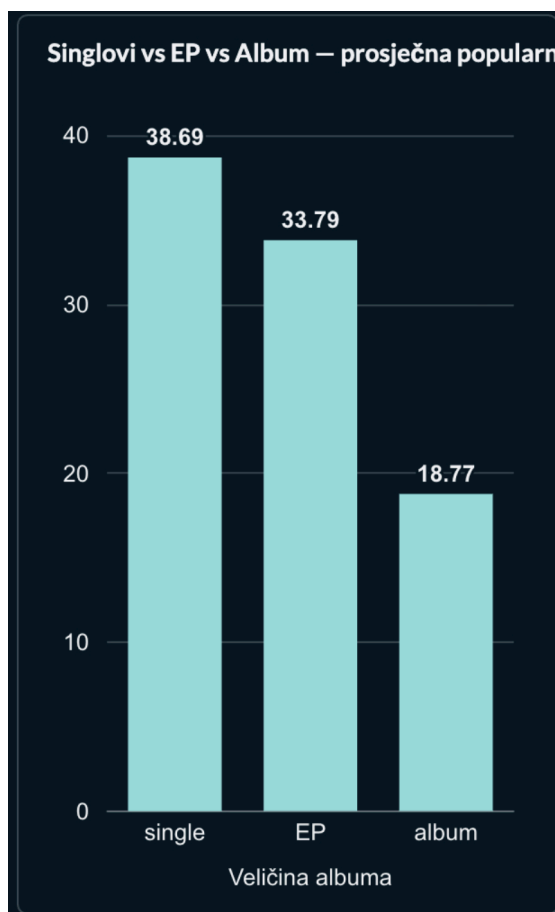
Korišten je grupirani bar chart koji za tri tempo kategorije (slow, mid, fast) prikazuje prosječnu glasnoću, ukupan broj pjesama i prosječnu popularnost. Može se vidjeti da pjesme s brzim tempom (fast) imaju najveću prosječnu popularnost, dok pjesme srednjeg tempa (mid) imaju najveći broj pjesama. Spori tempo (slow) bilježi najniže vrijednosti kod sve tri mjere: najtišu glasnoću, najmanji broj pjesama te najnižu prosječnu popularnost). Ovaj graf također potvrđuje da kod mlađe publike koja koristi platforme poput Spotify-a vole brze i živahne pjesme, njih nema puno ali imaju veliku penetraciju među publikom.

Top 15 žanrova po prosječnoj popularnosti



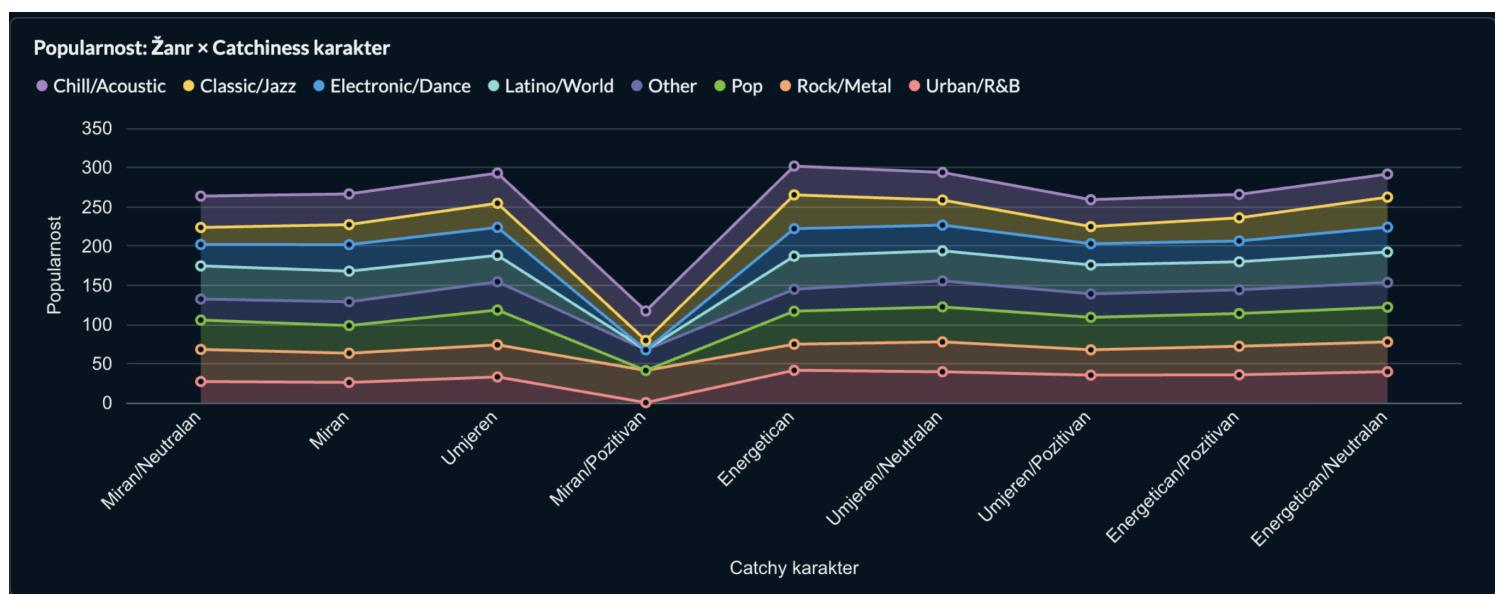
Korišten je horizontalni bar chart koji prikazuje petnaest žanrova s najvećom prosječnom popularnošću. Može se vidjeti da žanr k-pop ima najveću prosječnu popularnost (59,13), dok deep-house zatvara ljestvicu s 45,45. Ostali visokorangirani žanrovi uključuju pop-film (58,75) i metal (53,73). Ovdje je očito kako se radi o *snapshotu* iz 2023-2024 kada su navedeni žanrovi bili na svom vrhuncu.

Singlovi vs EP vs album – prosječna popularnost



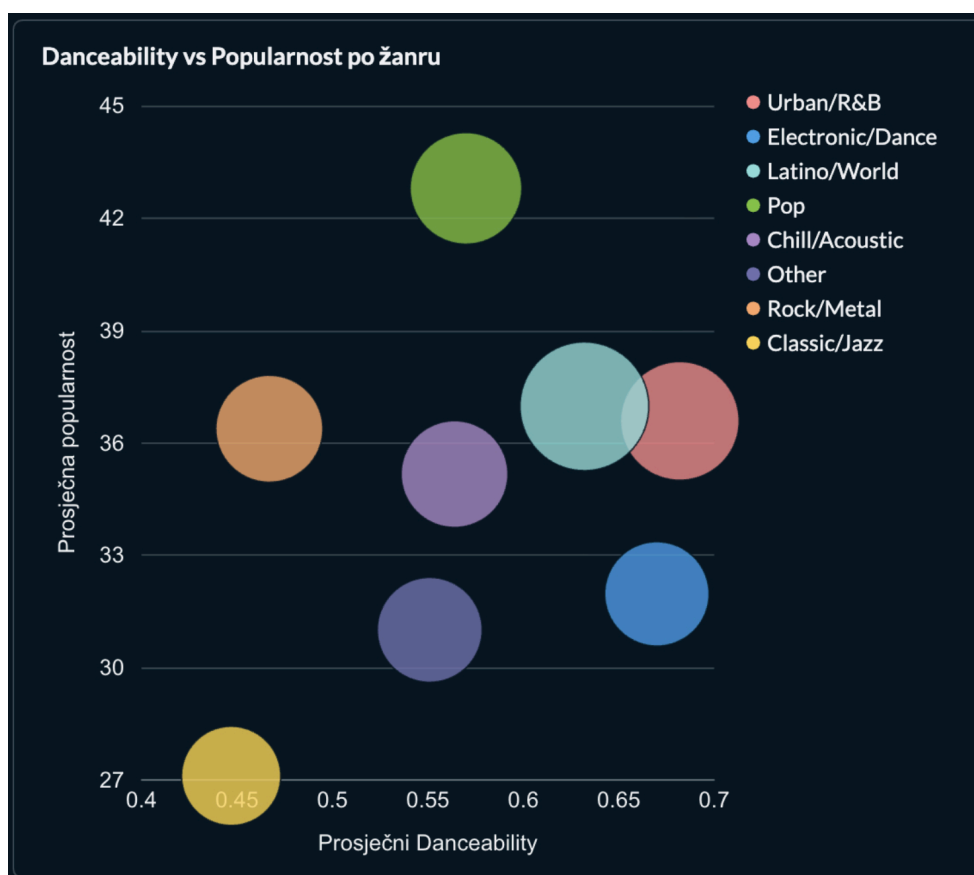
Korišten je bar chart koji uspoređuje prosječnu popularnost pjesama prema veličini albuma (singl, EP, album). Može se vidjeti da singlovi imaju najveću prosječnu popularnost (38,69), zatim EP (33,79), dok albumi imaju znatno nižu prosječnu popularnost (18,77). U glavnom ispalti se izdati single umjesto velikog albuma ukoliko je umjetnik siguran da je pjesma dobra.

Popularnost prema žanru i catchiness karakteru



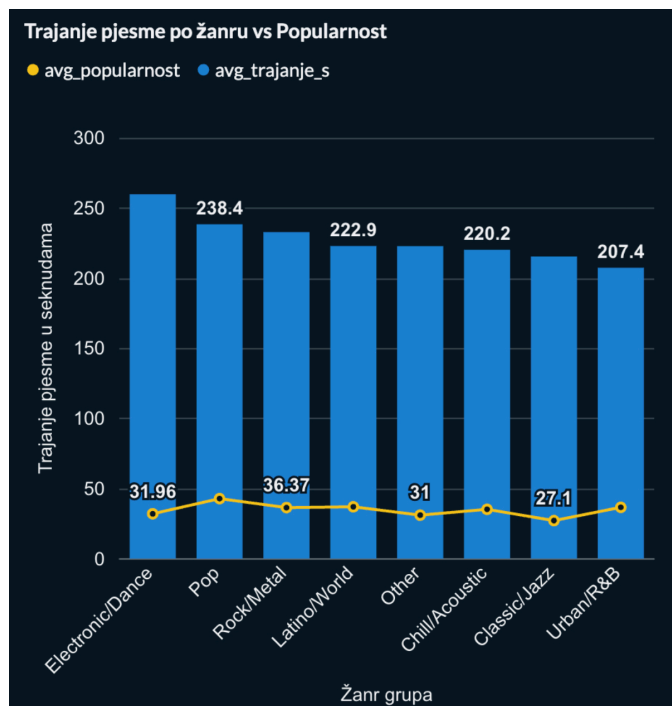
Korišten je grupirani bar chart koji na x-osi prikazuje devet kategorija catchiness karaktera (od mirnih preko umjerenih do energičnih), a na y-osi prosječnu popularnost u rasponu 0–350. Svaka žanr grupa predstavljena je zasebnom bojom. Može se vidjeti da žanrovi Electronic/Dance i Pop postižu najveću popularnost kod energičnih i pozitivnih kategorija (Energetičan/Pozitivan, Energetičan), dok Chill/Acoustic i Classic/Jazz imaju više vrijednosti kod mirnijih kategorija (Miran/Neutralan, Miran). Urban/R&B i Rock/Metal pokazuju ujednačeniju raspodjelu po svim razinama catchinessa. Zaključak ovog grafa je taj da ljudi kada slušaju Spotify ne slušaju “dosadne” ili “mirne” pjesme nego nešto energičnije od toga.

Odnos danceabilityja i popularnosti po žanr grupama



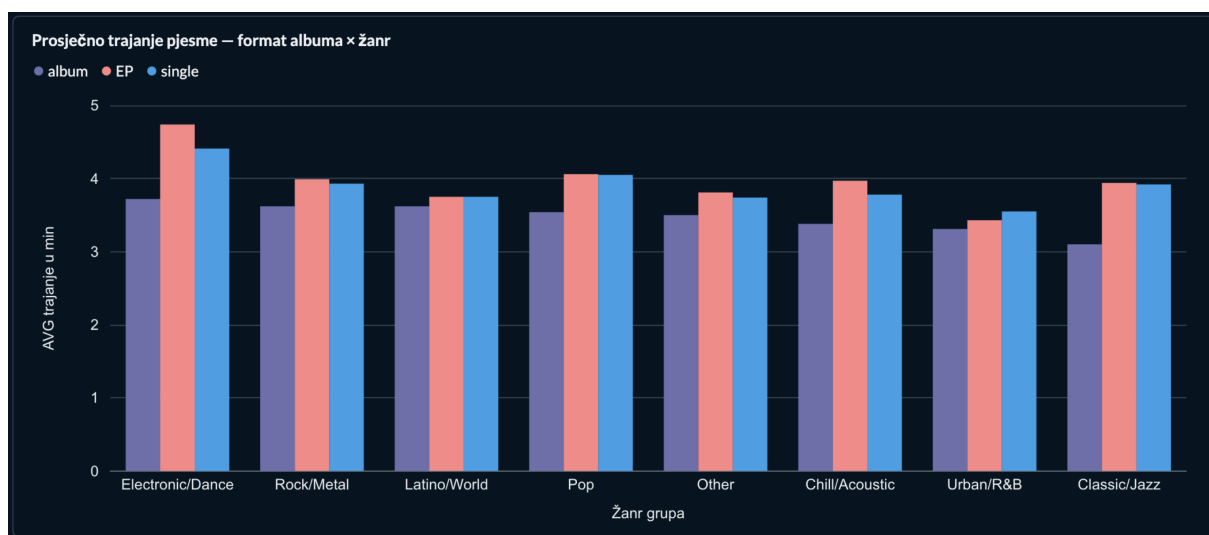
Korišten je scatter plot na kojem svaka točka predstavlja jednu žanr grupu, s prosječnim *danceabilityjem* na x-osi (raspon 0,4–0,7) i prosječnom popularnošću na y-osi. Može se vidjeti da žanrovi s višim *danceabilityjem* poput Urban/R&B i Electronic/Dance (oko 0,65–0,7) postižu relativno visoku popularnost, dok Classic/Jazz s najnižim *danceabilityjem* (ispod 0,45) ima i nisku popularnost. Pop i Latino/World nalaze se u sredini s umjerenim *danceabilityjem* i visokom popularnošću. Rock/Metal i Chill/Acoustic grupirani su uz niži *danceability* (0,5–0,55) i nižu do srednju popularnost. Zaključno, ne postoji strogo linearna veza, ali uočljiv je trend da veća *danceability* donekle korelira s većom popularnošću, uz iznimke poput Popa koji ima visoku popularnost uz tek nešto iznadprosječan *danceability*.

Trajanje pjesme po žanru u odnosu na popularnost



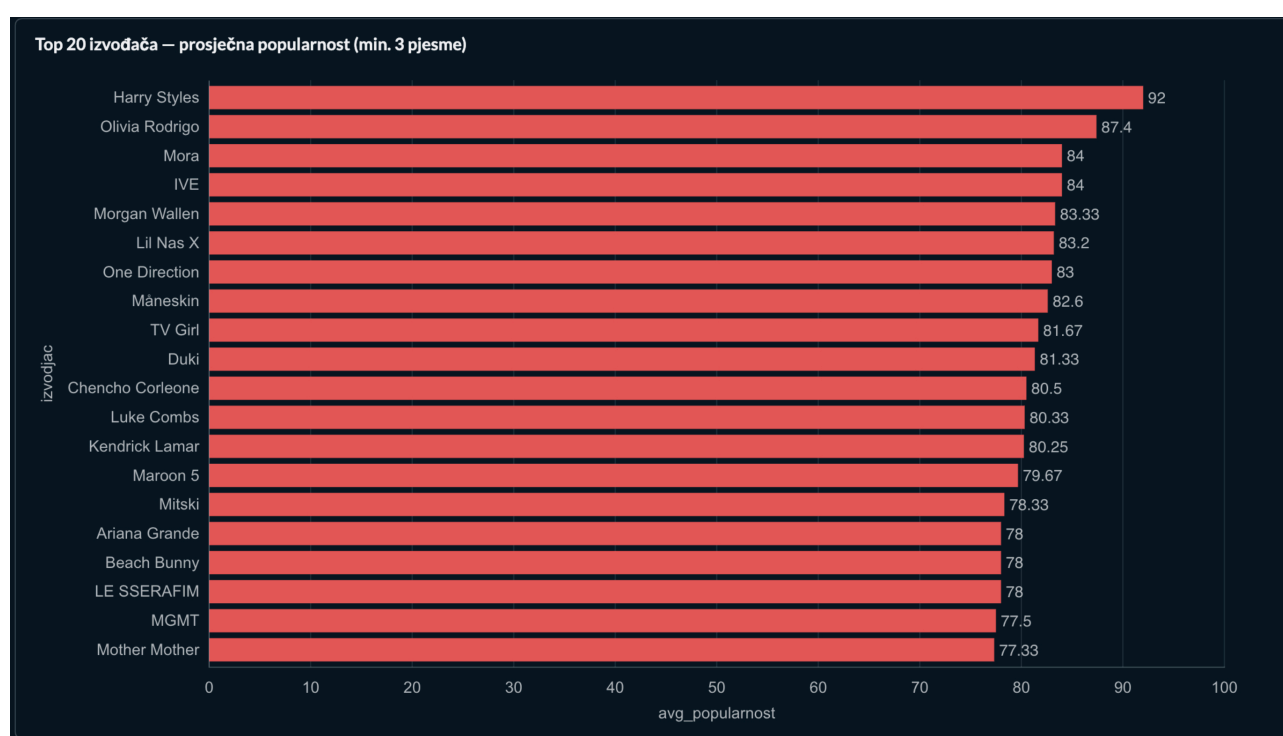
Korišten je grupirani bar chart koji za svaku žanr grupu prikazuje prosječnu popularnost i prosječno trajanje pjesme u sekundama. Može se vidjeti da žanrovi Pop, Rock/Metal i Latino/World imaju najveću prosječnu popularnost (36,37) uz najduže trajanje (238,4 sekunde), dok Classic/Jazz i Urban/R&B bilježe najnižu popularnost (27,1) i najkraće trajanje (207,4 sekunde). Electronic/Dance ima malo nižu popularnost (31,96) ali jednako dugo trajanje kao i najpopularniji žanrovi. Other i Chill/Acoustic nalaze se u sredini po obje mjere (popularnost 31, trajanje 220,2 sekunde). Možemo zaključiti da žanrovi s dužim pjesmama uglavnom postižu veću popularnost, iako postoje iznimke poput Electronic/Dancea.

Prosječno trajanje pjesme prema tipu izdanja (album, EP, single) po žanr grupama



Korišten je grupirani bar chart koji za svaku žanr grupu prikazuje prosječnu trajanje pjesme za tri tipa izdanja: album, EP i single. Može se vidjeti da Electronic/Dance ima najveće trajanje pjesme kod singlova i EP-ova, dok su albumi nešto niži. Pop i Rock/Metal također bilježe visoke vrijednosti, s tim da Pop ima najveće trajanje kod albuma. Najniže vrijednosti u svim kategorijama imaju Classic/Jazz i Urban/R&B, posebno Classic/Jazz s albumima. Zaključno, singlovi i EP-ovi u pravilu su duži od albuma unutar većine žanrova, osim kod Popa gdje su razlike manje izražene. Možemo zaključiti da duljina pjesme i u kojem stilu će biti izdana nije toliko bitna za krajnu popularnost pjesme.

Prikaz najpopularnijih umjetnika



Ovaj horizontalni barchart je samo napravljen iz moje osobne znatiželje da vidim što su ljudi u tom određenom periodu slušali. Ne daje nikakve značajnije *insighte* u analizi.

ZAKLJUČAK ANALIZE I PREPORUKA ZA STUDIO

Na temelju provedene analize mogu se izvući sljedeći zaključci i preporuke za Studio d.o.o. Singlovi su popularniji od albuma. Prosječna popularnost singlova iznosi 38,69, dok EP-ovi imaju 33,79, a albumi tek 18,77. Preporučuje se objavljivati više singlova nego cjelovitih albuma, točnije stavljati pjesme pred publiku što frekventnije kako bi neka “zakačila”.

Pjesme s eksplicitnim sadržajem (NSFW, 18+) jesu popularnije od čistih, ali razlika nije velika – 36,89 prema 32,85. Nije ključno inzistirati na eksplicitnosti. Treba izbjegavati „mirne“ pjesme. Žanrovi poput Chill/Acoustic i Classic/Jazz s mirnijim catchiness karakterom imaju nižu popularnost u odnosu na energičnije žanrove. Mali danceability gotovo uvijek znači malu popularnost. Classic/Jazz s najnižim danceabilityjem (ispod 0,45) ima i najnižu popularnost, dok Urban/R&B i Electronic/Dance s visokim danceabilityjem postižu visoke vrijednosti.

Pop je generalno najbolji izbor za studio zbog četiri razloga:

1. Najbolji omjer popularnosti i danceabilityja – pogodan za klubove i streaming servise.
2. Prosječno trajanje pjesme idealno je za suvremene playliste.
3. Najsigurniji je izbor – dosljedno visoka popularnost kroz sve analizirane metrike.
4. Tržište nije preplavljeno – samo 7,8% pjesama od svih Pop žanru, što ostavlja prostor za nove produkcije bez zasićenja.

Zaključno, za maksimalnu uspješnost preporučuje se produkcija pop singlova s umjerenom eksplicitnošću, visokom plesnošću i energičnim, a ne mirnim karakterom.

6. Zaključak

Jedna od najvažnijih sposobnosti u današnjem poslovnom svijetu jest donošenje odluka utemeljenih na podacima. Tržište se neprestano mijenja, ukusi publike evoluiraju, a ono što je bilo popularno jučer, danas više ne mora prolaziti.

Kroz ovaj projekt proveden je cjelovit ETL postupak nad Spotifyjevim skupom podataka iz 2023./2024. godine. Prvo je izvršeno izdvajanje podataka iz dva različita izvora – MySQL baze i CSV datoteke – nakon čega je uslijedila najzahtjevnija faza: transformacija. Uz usklađivanje naziva atributa i tipova podataka, posebna pažnja posvećena je spajanju podataka iz različitih izvora te implementaciji sporo mijenjajuće dimenzije tipa 2 za tablicu izvođača. Na kraju su podaci uspješno učitani u novu MySQL bazu podataka prema star shemi.

Za potrebe vizualizacije korišten je Metabase, FOSS alat pokrenut putem Dockera. Spajanjem na bazu podataka izrađeni su grafikoni kojima su analizirani odnosi između popularnosti pjesama i niza čimbenika. Rezultati analize pružaju konkretne smjernice za fiktivni „Studio d.o.o.“. Pokazalo se da singlovi imaju znatno veću prosječnu popularnost od albuma, da pjesme s eksplicitnim sadržajem jesu nešto popularnije, ali razlika nije drastična, te da energične pjesme s visokom plesnošću u pravilu prolaze bolje od mirnih.

Pop žanr izdvojio se kao najsigurniji izbor zbog najboljeg omjera popularnosti i danceabilityja, optimalnog trajanja pjesme te činjenice da tržište njime nije zasićeno – samo 7,8% pjesama u datasetu pripada pop žanru.

Naravno, metode korištene u ovom projektu nisu jedine. Postoje brojni drugi pristupi izdvajanju, transformaciji i vizualizaciji podataka. No, ono što ovaj projekt pokazuje jest da pravilno vođenje podataka – od provjere kvalitete dataseta do ETL procesa i konačne analize – može pružiti kvalitetne uvide u trenutno stanje tržišta. Za „Studio d.o.o.“ navedneo znači da može donositi informirane odluke o tome koje pjesme producirati, kako ih oblikovati i na koji način povećati šanse za njihovu popularnost.

Mario Bariša, 4.FIPU @ UNIPU

Ak. 2025/2026.

7. Literatura

Merlin stranica kolegija -Skladišta i rudarenje podataka

GitHub stranica s stranice kolegija

YT videozapisi sa predavanja